TABLE OF CONTENTS

S.NO.	CONTENTS
1.	ABSTRACT
2.	INTRODUCTION
3.	OBJECTIVES
4.	LITERARTURE REVIEW
5.	METHODOLOGY
6.	BLOCK DIAGRAM
7.	HARDWARE DESIGN
8.	SOFTWARE DESIGN
9.	HARDWARE COMPONENTS
10.	SOFTWARE DEVELOPMENT
11.	DEVELOPMENT OF PROGRAMMINGENVIRONMENT
12.	SOURCE CODE
13.	RESULT AND DISPLAY
14.	CIRCUIT CONNECTION
15.	WORKING PRINCIPLE
16.	IMPLEMENTATION
17.	TESTING AND VALIDATION
18.	RESULTS AND DISSCUSSION

19.	CONCLUSION

1.Abstract

The Electronic Voting Machine (EVM) stands as a testament to the evolution of democracy, replacing the traditional ballot papers and boxes with a streamlined electronic device. At the heart of democracy lies the fundamental right to vote, and the EVM simplifies this process significantly.

Gone are the days of laboriously stamping ballot papers and folding them into prescribed methods before depositing them into the Ballot Box, a process prone to errors and time-consuming. Enter the electronic voting machine, which condenses all these steps into a single, efficient device—the ballot unit. This technological advancement revolutionizes the electoral process by eliminating the need for physical papers, stamps, and boxes. Instead, voters can cast their choices swiftly and accurately through the EVM. Moreover, with the advent of biometric identifiers, the reliability of the voting process receives a significant boost.

2.Introduction

Elections form the foundation of democratic societies, providing a structured mechanism through which citizens express their collective will by choosing representatives and making key decisions. The integrity, accuracy, and efficiency of any electoral process are vital for ensuring public confidence in governance systems. Historically, elections have been conducted using traditional paper ballots, which, despite their simplicity, have been associated with various limitations, including errors in vote counting, fraudulent practices, logistical challenges, and delays in result declaration. In response to these concerns, electronic voting systems have

emerged as an alternative, offering the promise of faster, more reliable, and tamper-resistant electoral processes.

Electronic Voting Machines (EVMs) are systems designed to automate the process of casting and counting votes electronically, minimizing human involvement in critical stages of the election. The primary objective of EVMs is to enhance efficiency, accuracy, and security while simplifying the voting process for users. Several countries and institutions have adopted electronic voting technologies to modernize their electoral systems, each with unique approaches tailored to their specific requirements and constraints.

Arduino board

In developing this project, a structured methodology was followed, encompassing requirement analysis, hardware and software selection, circuit design, software development, system integration, and testing under simulated voting scenarios. The report provides comprehensive documentation of the system architecture, component specifications, software logic, circuit schematics, implementation process, and performance evaluation.

3.Objectives

The primary objective of this project is to design, develop, and implement a reliable, secure, and user-friendly Electronic Voting Machine (EVM) using Arduino technology. The project aims to address several limitations commonly associated with traditional paper-based voting systems and demonstrate how microcontroller-based systems can simplify, secure, and speed up the voting process, especially in small-scale elections held within educational institutions, clubs, societies, and community organizations.

Voting is a fundamental pillar of any democratic process, and ensuring its credibility, accuracy, and efficiency is of utmost importance. Paper-based voting systems, though still widely used in many places, often involve time-consuming manual vote counting, increased chances of human error, risk of vote tampering, and logistical challenges in transporting and managing physical ballot papers. Recognizing these limitations, this project was conceived with the goal of creating a cost-effective and easy-to-operate electronic voting solution suitable for small election environments.

4.Literature Review

Electronic voting, often referred to as e-voting, is the process of using electronic means to cast and count votes in an election. Over the years, numerous systems and methodologies have been developed to automate and streamline the voting process. The aim of this literature review is to examine previous work and existing technologies related to voting systems, with particular emphasis on electronic and microcontroller-based voting machines. By reviewing these systems, this section seeks to identify their strengths, limitations, and the areas where

improvements are necessary — which this project aims to address.

Traditionally, voting has been conducted using paper ballots, where voters mark their choice manually, and votes are counted by hand. While this system is simple and transparent, it is highly susceptible to human error, ballot tampering, and vote miscounting, especially in elections involving a large number of voters. Counting delays and logistical challenges related to ballot printing, distribution, and storage are additional drawbacks of paper-based systems.

To overcome these challenges, mechanical voting machines were introduced in the early 20th century, employing levers or rotary switches to cast and record votes. Although these devices improved counting accuracy and reduced the risks associated with paper ballots, they were bulky, expensive, and difficult to transport. Moreover, maintenance and operation required trained personnel, making them less suitable for smaller or remote voting environments.

Researchers and hobbyists have explored the use of microcontroller-based EVM prototypes for educational, experimental, and small-scale applications. Microcontrollers like Arduino Uno, PIC, and Raspberry Pi are ideal for such projects because of their affordability, ease of programming, and versatility. Arduino-based voting machines typically involve push buttons for vote casting, a microcontroller for data processing and storage, and an LCD or LED display for user interaction and result display.

Several academic studies have demonstrated the feasibility of such systems. For example, a 2018 study by Singh et al. proposed an Arduino-based EVM capable of handling four candidates and incorporating a buzzer system for vote confirmation. Their results showed that microcontroller-based systems can achieve high accuracy in recording votes and presenting results immediately after voting concludes. Similarly, another study by Kumar and Verma in 2020 highlighted the use of biometric authentication alongside an Arduino-powered voting system, enhancing security by preventing unauthorized voting.

In summary, existing literature indicates that while microcontroller-based voting machines offer a promising solution for small-scale, low-budget elections, there remains ample scope for enhancement, particularly in terms of scalability, security, and user authentication. This project builds on these studies by designing a cost-effective, simple, and reliable voting machine using an Arduino Uno, push buttons, and an LCD, with the potential for future upgrades.

5.Methodology

The methodology for this project outlines the structured approach followed to successfully design, develop, implement, and test an Arduino-based Electronic Voting Machine (EVM). A systematic process was essential to ensure that the final prototype meets the desired objectives of accuracy, reliability, and ease of use. The methodology encompasses several sequential phases, including requirement analysis, hardware selection, circuit design, software development, system integration, and testing.

Requirement Analysis

The initial step in the methodology involved identifying the functional and non-functional requirements of the electronic voting system. The functional requirements included the ability for voters to cast a vote using a button press, accurate vote recording, preventing multiple votes in a single session, and displaying the results after voting concludes. Non-functional requirements focused on simplicity, affordability, low power consumption, and user-friendliness.

Circuit Design

The system's circuit diagram was drafted using a simulation tool, ensuring proper connection between the Arduino Uno and external components. Each push button was connected to a digital input pin with a pull-down resistor to maintain a stable low state when not pressed. The LCD was connected to digital pins with the help of an appropriate library to display messages and results. A buzzer was interfaced to provide feedback when a vote is cast.

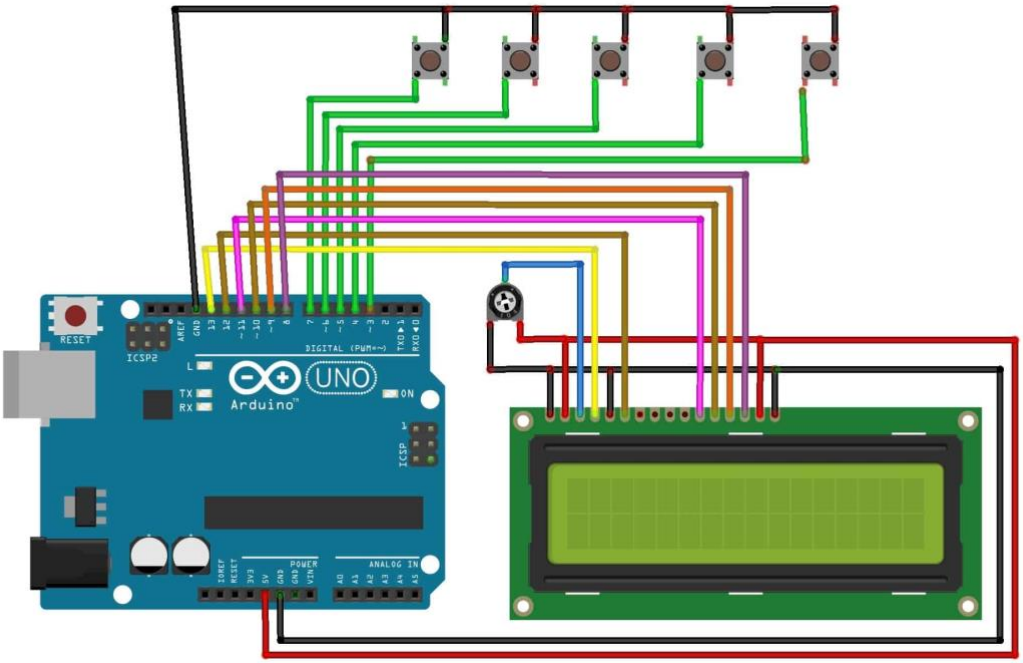
Software Development

The Arduino program was developed using the Arduino IDE. The code was written to initialize the LCD display, configure input and output pins, and maintain vote counts for each candidate. Button debouncing techniques were applied in the code to avoid registering multiple counts from a single press. A function was created to check button states, increment vote counters, activate the buzzer briefly, and update the LCD.

System Integration

After completing the hardware assembly and software development, both were integrated. The components were connected according to the circuit diagram, and the program was uploaded to the Arduino Uno. The system was powered and tested for proper functioning.

6.Block Diagram



7.Hardware Design

The hardware design of the EVM consists of the physical components responsible for capturing votes, recording data, and displaying results. The major hardware components include the Arduino Uno microcontroller, push buttons, a 16x2 Liquid Crystal Display (LCD), a buzzer, and connecting wires.

Arduino Uno was selected as the system's control unit due to its affordability, ease of programming, and sufficient number of digital I/O pins for interfacing with the required components. The push buttons are used by voters to cast their votes. Each button is assigned to a specific candidate. When a button is pressed, it sends a signal to the corresponding digital input pin on the Arduino.

To display instructions and election results, a 16x2 LCD display is connected to the Arduino. This LCD can show two lines of text with up to 16 characters per line, making it ideal for displaying candidate names, voting confirmations, and final results. A buzzer is also included to provide auditory feedback whenever a vote is successfully cast, enhancing user interaction and confirming that the system has registered their input.

8.Software Design

The software, or program design, involves developing the logic and control algorithms that govern how the hardware behaves. The program was written in C/C++ using the Arduino IDE.

The program begins by initializing all required components, including the LCD display and digital I/O pins. Each push button is configured as an input pin, while the buzzer is set as an output. The software continuously monitors the state of each push button using a loop structure. When a button press is detected, the corresponding vote counter is incremented by one, the buzzer is activated briefly, and a confirmation message is displayed on the LCD.

Once the voting process is complete, an administrator can press a dedicated “Result” button to display the final vote tally for each candidate on the LCD. The program is structured to prevent any further voting after the result has been declared, ensuring the integrity of the voting session.

Integration Design

The integration of hardware and software was achieved by carefully connecting the components as per the circuit diagram and uploading the developed code to the Arduino Uno. Proper system integration ensures that the hardware responds accurately to the program’s logic and delivers a reliable voting experience.

9.Hardware Components

The hardware components selected for this Arduino-based Electronic Voting Machine (EVM) are crucial for ensuring that the system functions reliably, efficiently, and securely. The main components of the system include the Arduino Uno microcontroller, push buttons, 16x2 Liquid Crystal Display (LCD), buzzer, and associated circuit components such as resistors, wires, and breadboards. Each of these components was chosen based on their compatibility, ease of use, and suitability for a small-scale, cost-effective voting machine.

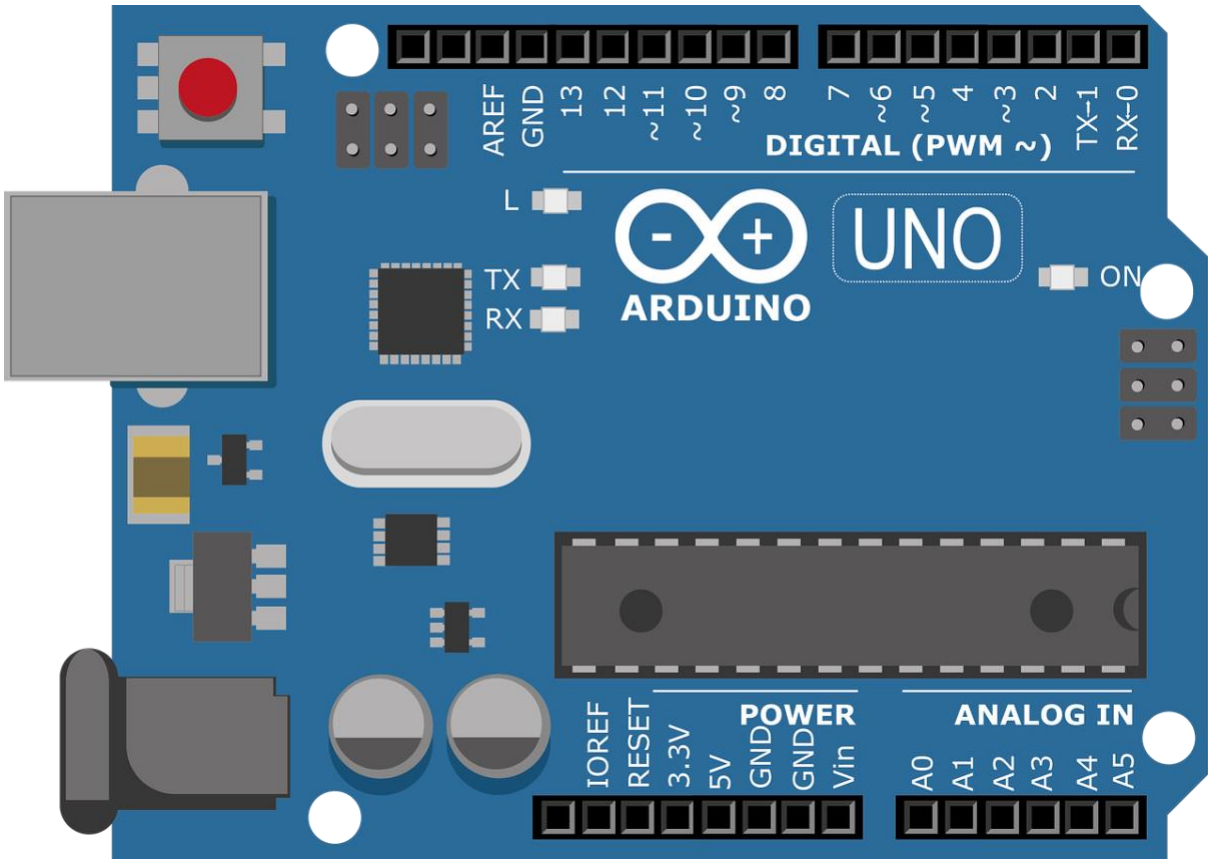
Arduino Uno Microcontroller

At the heart of the voting machine is the Arduino Uno, a microcontroller board based on the ATmega328P. The Arduino Uno is a popular open-source development platform widely used for prototyping and educational projects. It was selected for this project due to its:

Affordability: Arduino boards are inexpensive, making them ideal for small-scale projects like this one.

Ease of programming: The Arduino IDE simplifies the process of writing, uploading, and testing code.

I/O versatility: The Arduino Uno features 14 digital I/O pins and 6 analog input pins, which are more than sufficient for interfacing with the buttons, LCD, and buzzer.



Push Buttons

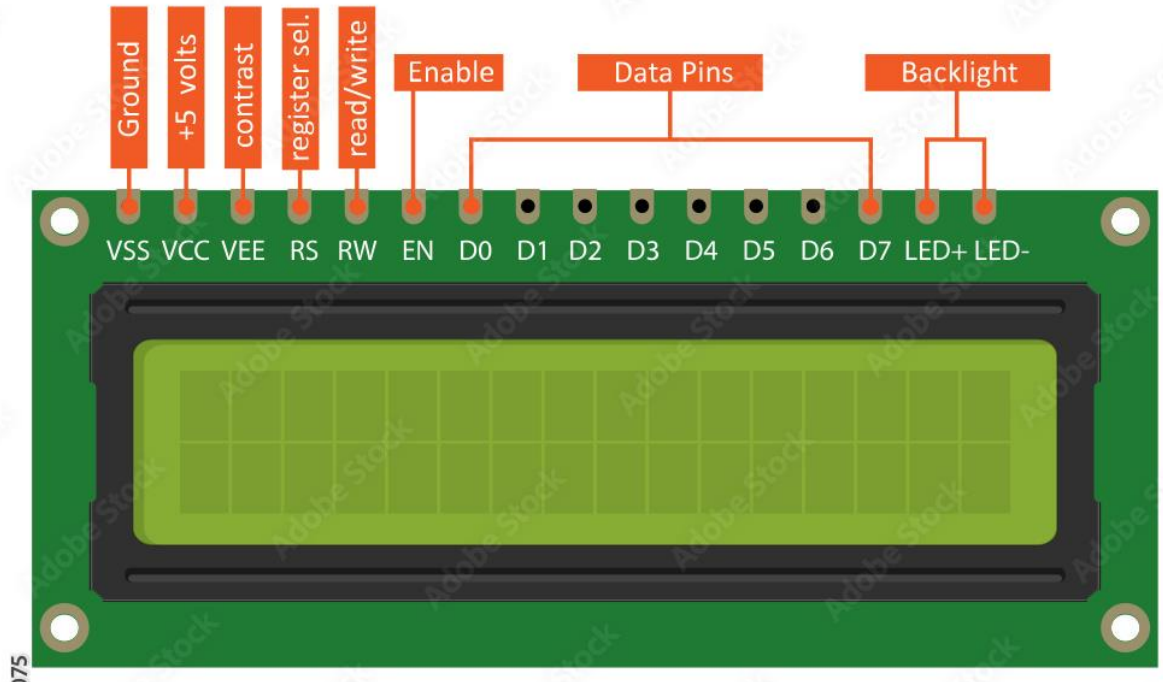
The push buttons are used by voters to cast their votes. Each button is assigned to a specific candidate, and pressing a button increments the vote count for that candidate. In this system, four push buttons were used, each representing a different candidate.

16x2 Liquid Crystal Display (LCD)

A 16x2 LCD display is used to present information to the voter and election administrator. The display consists of two rows, each capable of displaying

up to 16 characters, making it ideal for showing candidate names, instructions, voting confirmation, and final results.

The LCD is connected to the Arduino via a set of digital pins. It uses the LiquidCrystal library, which simplifies the process of displaying text. The screen displays real-time information, such as a prompt for voters to select a candidate, as well as the final results after the voting session ends. The display also provides feedback when a vote is successfully recorded.



Buzzer

The buzzer is integrated into the system to provide auditory feedback when a vote is successfully cast. Upon pressing a button, the buzzer emits a brief sound to confirm to the voter that their vote has been recorded. This auditory confirmation ensures that voters know their action has been successfully registered, enhancing the overall user experience.

The buzzer is connected to one of the digital output pins on the Arduino. When a button press is detected, the program triggers the buzzer to make a short beep. The buzzer can be activated using the digitalWrite() function in the Arduino code.

Power Supply

The Arduino Uno is powered via a 5V DC adapter or through a USB connection from a computer. The components such as the push buttons, LCD, and buzzer all operate within the 5V range, making them compatible with the Arduino Uno's power requirements. Powering the system via USB also allows for easy programming and debugging during development.

10.Software Development

The software development phase for the Arduino-based Electronic Voting Machine (EVM) is critical to ensure that the system functions efficiently and accurately. The software serves as the interface between the hardware components and the user, managing actions such as vote recording, result tallying, and providing feedback to the user through auditory and visual cues. This section outlines the software's development process, key features, and structure, as well as the challenges encountered during implementation.

11.Developement of Programming Environment

The software was developed using the Arduino Integrated Development Environment (IDE), which supports the C and C++ programming languages. The Arduino IDE offers an easy-to-use platform for writing, debugging, and uploading code to the Arduino Uno microcontroller. The choice of C/C++ is ideal for embedded systems like Arduino due to their efficiency in handling low-level operations and interaction with hardware components.

The software interacts directly with hardware components like push buttons, LCD displays, and the buzzer. The code was designed to be simple and modular, facilitating easy understanding and debugging while ensuring real-time responsiveness to user input.

System Initialization

Upon powering up the system, the program begins with an initialization phase. In this phase, the LCD display is initialized to show a welcome message or instructions for the voter. The digital input pins connected to the push buttons are configured, and each button is assigned to a specific candidate. Additionally, the buzzer is activated briefly to signal that the system is ready to accept input.

Voting Logic

The core functionality of the EVM software lies in the voting logic. The system continuously monitors the state of the push buttons. When a voter presses a button corresponding to their candidate of choice, the following steps occur:

1. Button Press Detection: The program checks the state of each push button, detecting when a button is pressed (a change from LOW to HIGH signal). Button debouncing is implemented to prevent multiple counts for a single press. This is achieved by checking the button state after a short delay to ensure it has been pressed once and is stable.

2. Vote Registration: Once the button press is detected and debounced, the corresponding vote counter for the selected candidate is incremented. The program keeps track of the vote counts for each of the four candidates in separate variables.

3. User Feedback: After the vote is registered, the buzzer is activated to provide auditory confirmation that the vote has been counted. The LCD display is updated to show a message such as “Vote Registered” or “Vote Counted,” letting the voter know that their action has been successfully recorded.

Result Display and Finalization

Once the voting session has concluded, the system enters the result display phase. To ensure transparency, the results can be displayed on the LCD. The administrator can press a dedicated button to trigger the display of final results.

12.Source code

The core of the program is contained within the loop() function, which continuously checks the system’s state. The loop ensures that the system is always listening for button presses and updating the LCD and buzzer as needed. The software operates in real-time, processing votes as they occur and updating feedback immediately.

```
#include<LiquidCrystal.h>
LiquidCrystal lcd(13, 12, 11, 10, 9, 8);

#define S1 7
#define S2 6
#define S3 5
#define S4 4
#define S5 3
int vote1=0;
int vote2=0;
int vote3=0;
int vote4=0;
void setup()
```

```
{
pinMode(S1, INPUT);
pinMode(S2,INPUT);
pinMode(S3,INPUT);
pinMode(S4,INPUT);
pinMode(S5,INPUT);
lcd.begin(16, 2);
lcd.print(" Electronic ");
lcd.setCursor(0,1);
lcd.print(" Voting Machine ");
delay(4000);
digitalWrite(S1, HIGH);
digitalWrite(S2, HIGH);
digitalWrite(S3, HIGH);
digitalWrite(S4, HIGH);
digitalWrite(S5, HIGH);
lcd.clear();
lcd.setCursor(1,0);
lcd.print("A");
lcd.setCursor(5,0);
lcd.print("B");
lcd.setCursor(9,0);
lcd.print("C");
lcd.setCursor(13,0);
lcd.print("D");
}
void loop()
{
lcd.setCursor(1,0);
lcd.print("A");
lcd.setCursor(1,1);
lcd.print(vote1);
lcd.setCursor(5,0);
lcd.print("B");
lcd.setCursor(5,1);
lcd.print(vote2);
lcd.setCursor(9,0);
lcd.print("C");
lcd.setCursor(9,1);
lcd.print(vote3);
lcd.setCursor(13,0);
lcd.print("D");
```

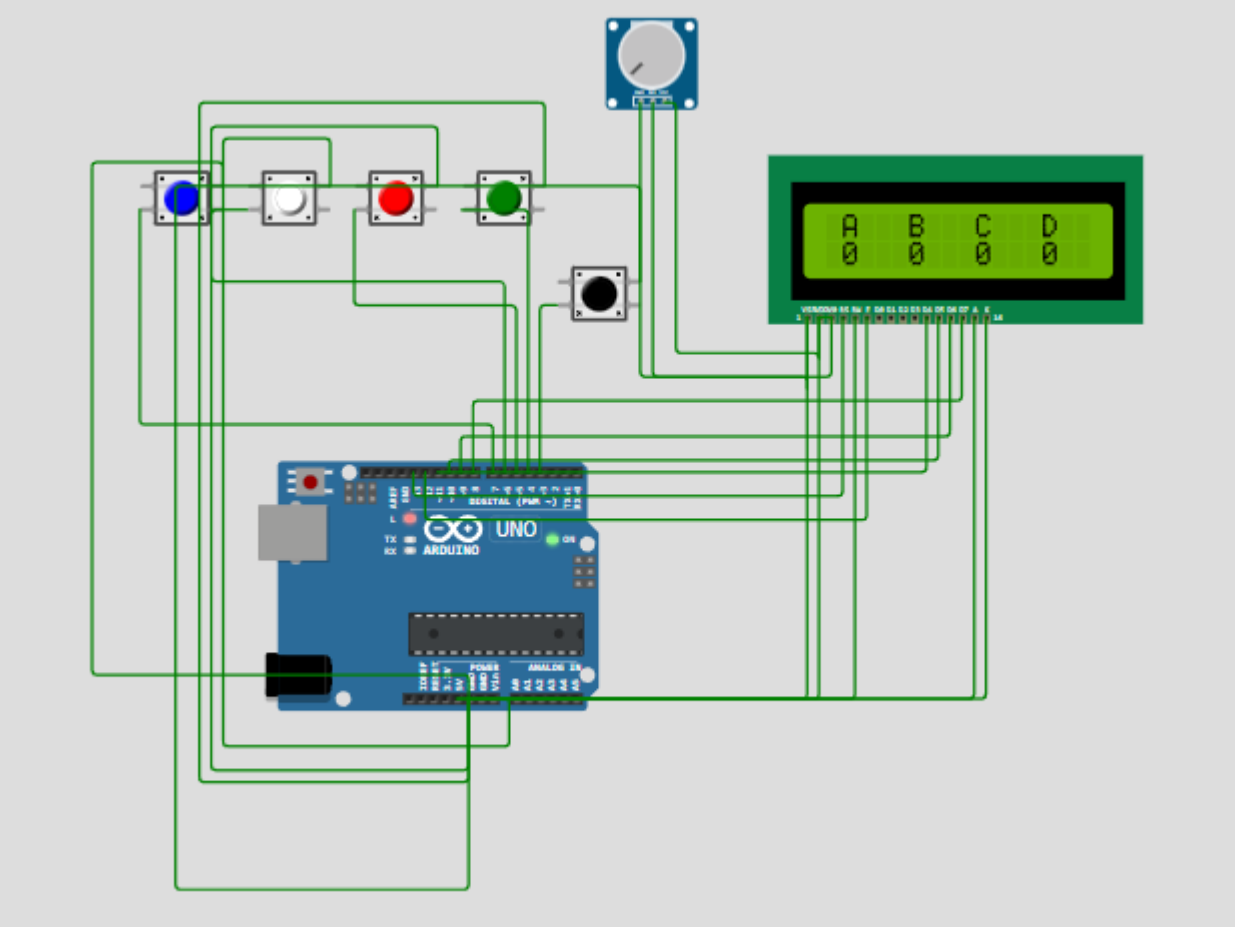
```
lcd.setCursor(13,1);
lcd.print(vote4);
if(digitalRead(S1)==0)
vote1++;
while(digitalRead(S1)==0);
if(digitalRead(S2)==0)
vote2++;
while(digitalRead(S2)==0);
if(digitalRead(S3)==0)
vote3++;
while(digitalRead(S3)==0);
if(digitalRead(S4)==0)
vote4++;
while(digitalRead(S4)==0);
if(digitalRead(S5)==0)
{
int vote=vote1+vote2+vote3+vote4;
if(vote)
{
if((vote1 > vote2 && vote1 > vote3 && vote1 > vote4))
{
lcd.clear();
lcd.print("A is Winner");
delay(3000);
lcd.clear();
}
else if((vote2 > vote1 && vote2 > vote3 && vote2 > vote4))
{
lcd.clear();
lcd.print("B is Winner");
delay(3000);
lcd.clear();
}
else if((vote3 > vote1 && vote3 > vote2 && vote3 > vote4))
{
lcd.clear();
lcd.print("C is Winner");
delay(3000);
lcd.clear();
}
else if(vote4 > vote1 && vote4 > vote2 && vote4 > vote3)
{
```

```
lcd.setCursor(0,0);
lcd.clear();
lcd.print("D is Winner");
delay(3000);
lcd.clear();
}
 else if(vote4 > vote1 && vote4 > vote2 && vote4 > vote3)
{
lcd.setCursor(0,0);
lcd.clear();
lcd.print("D is Winner");
delay(3000);
lcd.clear();
}

else
{
lcd.clear();
lcd.print(" Tie Up Or ");
lcd.setCursor(0,1);
lcd.print(" No Result ");
delay(3000);
lcd.clear();
}

}
else
{
lcd.clear();
lcd.print("No Voting....");
delay(3000);
lcd.clear();
}
vote1=0;vote2=0;vote3=0;vote4=0,vote=0;
lcd.clear();
}

}
```

13.Result Display

Once voting is complete, an administrator button can be pressed to display the final vote counts. The Arduino retrieves the values from the vote counter variables and presents them on the LCD display, typically showing each candidate's name alongside their total vote count.

After results are displayed, the program can disable further voting by locking the buttons or restarting the system. This prevents unauthorized voting after the official closing time.



Components Overview

The following components are used in the circuit:

1. Arduino Uno Microcontroller – Acts as the brain of the voting machine, controlling all components based on the input it receives and executing the corresponding logic.

2. Push Buttons – Used for voting, with each button representing a candidate. Each button is connected to one of the digital input pins on the Arduino.

3. 16x2 LCD Display – Displays the voting instructions, status, and final results

4. Buzzer – Provides auditory feedback to the voter upon casting a vote.

5. Resistors – Used for pull-down configuration to stabilize button signals.

6. Jumper Wires – Connect all components together on a breadboard or through a custom PCB.

14.Circuit Connection

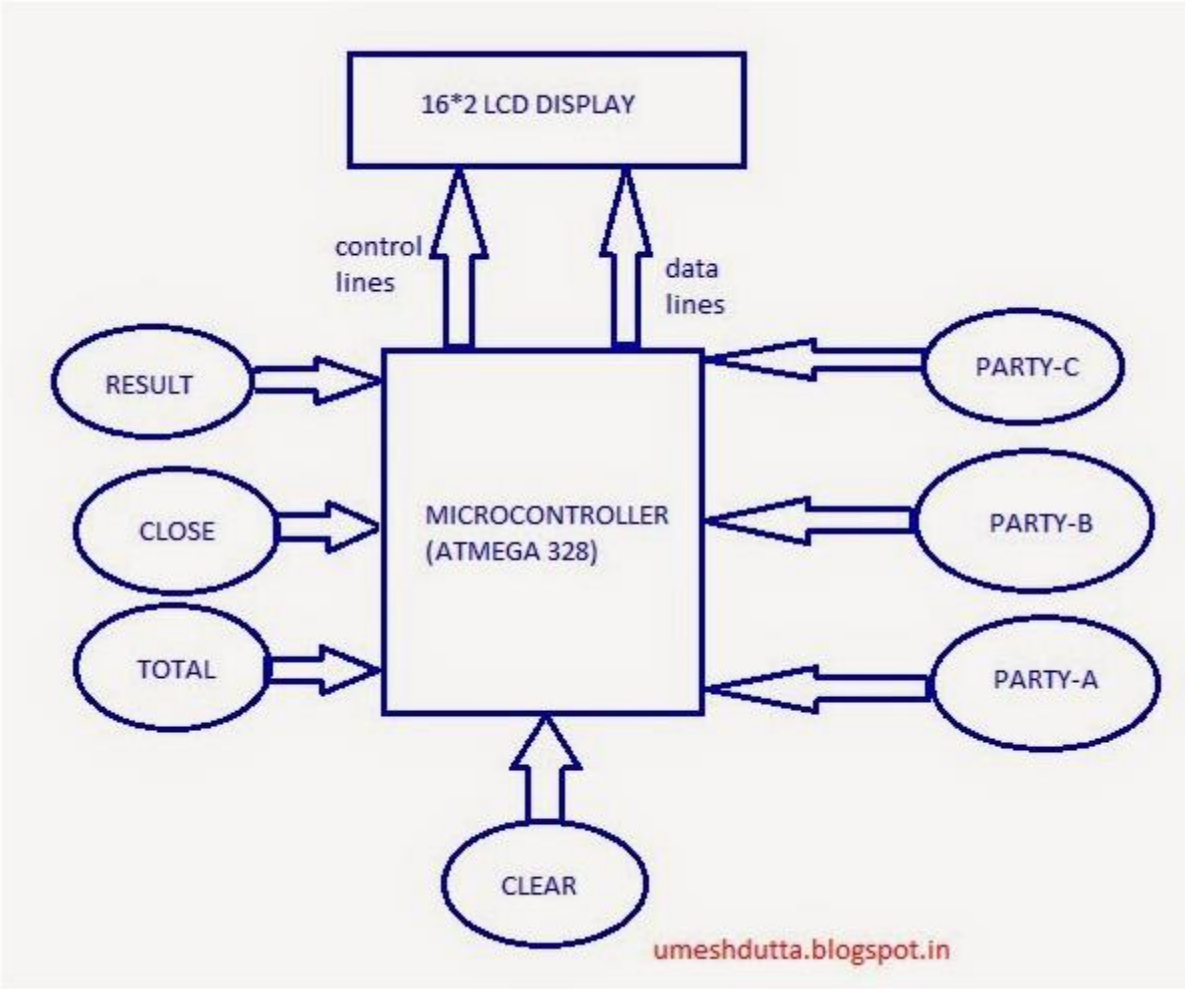
The Arduino

Uno has a total of 14 digital I/O pins, six of which can be used as analog inputs. In this project, several of these pins are used for button inputs, the LCD connections, and controlling the buzzer.

Digital Pins 2-5These are connected to the four push buttons. Each button is wired with a pull-down resistor (typically 10kΩ) to ensure the pin reads LOW when not pressed and HIGH when pressed. This pull-down resistor keeps the signal stable and prevents erratic behavior.

Pin 12 (Buzzer): The buzzer is connected to pin 12 of the Arduino. A current-limiting resistor may also be placed in series with the buzzer to prevent excessive current from damaging the components.

Push Buttons Each of the four push buttons is connected to a digital input pin on the Arduino. One terminal of each button is connected to the digital pin, and the other terminal is connected to the ground (GND). In between, a pull-down resistor is placed to ensure that the default state of the button is LOW (no press), while pressing the button changes the state to HIGH (button pressed). This is essential to prevent any floating input signals, which can lead to erratic behavior.



15.Working Principle

The working principle of the Arduino-based Electronic Voting Machine (EVM) revolves around the basic concept of registering a vote through a physical input device (push button), confirming the action, and securely recording the result. The system is designed to be simple, cost-effective, and reliable, primarily intended for small-scale voting events such as college elections, community polls, or prototype demonstrations. This section describes how each part of the system functions and interacts, ensuring an accurate and transparent voting process.

Overview of Operation

At its core, the Electronic Voting Machine works by detecting button presses corresponding to individual candidates, incrementing their

respective vote counters, and providing immediate feedback to the voter. It also displays the final vote tally upon request. The working principle can be divided into several sequential stages:

1. System Initialization
2. Vote Casting
3. Vote Confirmation
4. Vote Counting and Storage
5. Result Display

Each stage is crucial in ensuring the accuracy and reliability of the overall voting process.

System Initialization

When the EVM is powered on, the Arduino microcontroller initializes all components. It configures the digital input pins connected to the push buttons and the output pins linked to the LCD display and buzzer. The LCD screen displays a welcome message or instructions for the voter. Simultaneously, vote counters for all candidates are reset to zero.

This initialization ensures that the system starts in a known state, free from residual data or erroneous counts from previous sessions.

To prevent the issue of multiple votes being counted due to mechanical contact bounce — a phenomenon where the button makes and breaks contact rapidly when pressed — a debouncing technique is implemented in the software. This involves introducing a short delay and rechecking the button's state before confirming a valid vote.

Vote Confirmation

Upon successful detection of a valid button press, the system performs three actions:

Increments the vote counter associated with the selected candidate.

Activates the buzzer for a short beep, providing auditory feedback to the voter that their vote has been successfully recorded.

Displays a message on the LCD screen such as “Vote Registered” or “Thank you for voting.”

This combination of feedback mechanisms ensures that the voter knows their action has been recorded correctly and that no further input is required from them.

Vote Counting and Storage

Each candidate has a dedicated variable in the Arduino’s memory that keeps track of the number of votes received. Every time a corresponding button is pressed, its vote counter increments by one. Since the Arduino’s microcontroller uses volatile memory (RAM), vote counts are held during the session and can be displayed at any time until the system is powered off.

16.Implementation

The implementation phase of the Arduino-based Electronic Voting Machine (EVM) project involves assembling the hardware components, programming the microcontroller, testing the system’s functionality, and ensuring smooth interaction between the software and hardware. This stage transforms the theoretical design into a working prototype capable of performing electronic voting operations reliably and efficiently. The implementation process follows a structured approach to integrate, test, and validate each part of the system, ensuring the EVM works as intended

Software Installation and Uploading Code

After the hardware assembly, the Arduino IDE (Integrated Development Environment) is installed on a computer for programming the Arduino Uno microcontroller. The code, written in C/C++, includes commands to detect button presses, register votes, provide feedback through the LCD and buzzer, and display the final results.

The software program is uploaded to the Arduino via a USB cable. The program includes proper debouncing for the push buttons, ensuring that each button press is registered as a single vote, and it also includes clear logic for displaying voting instructions, vote confirmation, and final results on the LCD.

17.Testing and Validation

The Testing and Validation phase is a crucial part of any electronic or software-based system development, ensuring that the final product performs as intended under different conditions. For the Arduino-based Electronic Voting Machine (EVM), extensive testing was carried out on both individual components and the fully integrated system to confirm its reliability, accuracy, and stability. This section explains the structured testing strategy, methods applied, observations recorded, and validation procedures undertaken to ensure the EVM met its design specifications.

Testing Objectives

The primary objectives of testing this EVM system included:

Verifying that each vote button accurately registers a single vote.

Ensuring the LCD displays correct messages and vote counts.

Confirming the buzzer provides feedback for every successful vote.

Testing system response under normal and boundary conditions.

Identifying and eliminating any issues such as button bounce, miscounts, or system hang-ups.

Validating result tallying accuracy after multiple voting sessions.

18.Results and Discussion

The Results and Discussion section presents the findings from the implementation and testing phases of the Arduino-based Electronic Voting Machine (EVM) project. It also analyzes the significance of these results, discusses the effectiveness of the system, identifies any limitations encountered, and suggests possible improvements for future enhancements. The results confirm the accuracy, functionality, and efficiency of the voting machine for small-scale, secure, and reliable voting applications.

Results Overview

After assembling the EVM hardware and uploading the program code to the Arduino Uno microcontroller, multiple tests were conducted to validate the voting system's performance under different scenarios. The following results were consistently observed during various trial runs:

Accurate Vote Counting: Each button press for a candidate correctly incremented the associated vote count by one. There were no instances of missed or duplicate counts when proper debouncing was applied.

System Stability: The EVM operated smoothly during extended testing sessions and remained stable even after multiple consecutive votes and power interruptions.

19.Conclusion

In conclusion, the results confirm that the Arduino-based EVM operates reliably for small-scale applications. The system met all intended functional

requirements and proved effective in accurately recording and displaying votes. The discussion highlights the potential for further enhancements, especially in data retention, scalability, and security, to adapt this simple design for broader use cases.